

Credit Card Fraud Detection using Machine Learning

A Project Report submitted in partial fulfilment of the requirements for
the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Akshita Modi	AP24110010386
Lakshay	AP24110010677
Mohana Krishna Reddy Pittu	AP24110011262
Nadimpally Dheva Harshit	AP24110011553

Under the guidance of

Mr. Ajay Dilip Kumar Marapatla

Department of Computer Science and Engineering

SRM University - AP

Neerukonda, Mangalagiri Mandal

Guntur District, Andhra Pradesh, India

September 2026

CERTIFICATE

This is to certify that the project report entitled “**Credit Card Fraud Detection using Machine Learning**” is a bona fide record of the work carried out by **Akshita Modi (AP24110010386), Lakshay (AP24110010677), Mohana Krishna Reddy Pittu (AP24110011262) and Nadimpally Dheva Harshit (AP24110011553)**, students of the Department of Computer Science and Engineering, SRM University – AP, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The work presented in this report is original and has been carried out under my supervision. To the best of my knowledge, the contents of this report have not been submitted elsewhere for the award of any other degree or diploma.

Mr. Ajay Dilip Kumar Marapatla

Supervisor

Department of Computer Science and
Engineering

SRM University – AP

Head of the Department

Department of Computer Science
and Engineering

SRM University – AP

Place: Amaravati

Date:

DECLARATION

We hereby declare that the project report entitled “**Credit Card Fraud Detection using Machine Learning**” submitted to the Department of Computer Science and Engineering, SRM University - AP, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out by us under the supervision of Mr. Ajay Dilip Kumar Marapatla.

Every experimental result presented in this report was produced by code written for this project and is reproducible from the accompanying source repository. All figures and numerical values in Chapter 5 are generated directly by that code; none has been transcribed from any other source. The random seed used throughout is **77**, the last two digits of the roll number AP24110010677, as required.

We further declare that this work has not been submitted, in whole or in part, for the award of any other degree or diploma at this or any other institution.

Akshita Modi	AP24110010386
Lakshay	AP24110010677
Mohana Krishna Reddy Pittu	AP24110011262
Nadimpally Dheva Harshit	AP24110011553

Place: Amaravati

Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, **Mr. Ajay Dilip Kumar Marapatla**, Department of Computer Science and Engineering, SRM University – AP, whose teaching of the Machine Learning course forms the entire foundation of this project. Every algorithm implemented here was drawn from his lectures, and the project is in essence an attempt to apply all five units of the course to one real problem.

We are grateful to the Department of Computer Science and Engineering, SRM University – AP, for the academic environment and computational resources that made this work possible.

We thank the Machine Learning Group at the Université Libre de Bruxelles for making the credit card fraud detection dataset publicly available.

Finally, we thank our families and friends for their patience and encouragement throughout this work.

Akshita Modi	AP24110010386
Lakshay	AP24110010677
Mohana Krishna Reddy Pittu	AP24110011262
Nadimpally Dheva Harshit	AP24110011553

ABSTRACT

Credit card fraud causes substantial financial losses worldwide, and the extreme rarity of fraudulent transactions makes its automatic detection a demanding machine learning problem. This project applies the complete syllabus of the Machine Learning course – all five units – to the detection of fraudulent transactions in the benchmark dataset published by the Machine Learning Group of the Université Libre de Bruxelles, containing 284,807 transactions of which only 492 are fraudulent.

After removing 1,081 duplicate records, 283,726 transactions remain, 473 of them fraudulent, a prevalence of 0.1667%. At this imbalance a classifier that predicts “legitimate” for every transaction attains 99.8333% accuracy while detecting nothing, so accuracy is rejected as an evaluation metric and recall, precision, F1-score and the area under the precision-recall curve are used instead.

Twelve models were trained and evaluated on an identical stratified split with random seed 77: a decision tree using the ID3 entropy criterion, k-nearest neighbour, Gaussian naive Bayes, a support vector machine with an RBF kernel, a perceptron, a multilayer network trained by backpropagation, bagging, Random Forest, AdaBoost, XGBoost, a fixed-rule voting ensemble, a trained-rule stacking ensemble, and unsupervised K-means clustering. Supporting analyses cover the curse of dimensionality, univariate and multivariate feature selection, Principal Component Analysis, Linear Discriminant Analysis, and computational learning theory.

XGBoost performed best, achieving precision 0.9508, recall 0.8169 and an F1-score of 0.8788, detecting 116 of 142 fraudulent transactions in the test set while raising only 6 false alarms. Five-fold cross validation confirmed the ranking. Three further findings emerged from the experiments: the perceptron failed to converge on XOR exactly as the convergence theorem predicts; k-nearest neighbour performance declined as attributes were added, demonstrating the curse of dimensionality on real data; and the Bayes optimal decision threshold derived from an asymmetric loss matrix could only be applied after the naive Bayes posteriors were calibrated, since the derivation assumes genuine probabilities.

Keywords: Credit Card Fraud Detection, Machine Learning, Class Imbalance,

Decision Tree, ID3, k-Nearest Neighbour, Naive Bayes, Bayes Optimal Decision, Support Vector Machine, Perceptron, Backpropagation, Ensemble Learning, Rule Fusion, Clustering.

LIST OF ABBREVIATIONS

ANN	Artificial Neural Network
AP	Average Precision
CCFD	Credit Card Fraud Detection
CV	Cross Validation
DT	Decision Tree
FN	False Negative
FP	False Positive
ID3	Iterative Dichotomiser 3
k-NN	k-Nearest Neighbour
LDA	Linear Discriminant Analysis
MAP	Maximum A Posteriori
ML	Machine Learning
MLP	Multilayer Perceptron
M-P	McCulloch-Pitts
NB	Naive Bayes
PAC	Probably Approximately Correct
PCA	Principal Component Analysis
PR-AUC	Precision-Recall Area Under Curve
RBF	Radial Basis Function
RF	Random Forest
RFE	Recursive Feature Elimination
ROC-AUC	Receiver Operating Characteristic Area Under Curve
SVM	Support Vector Machine
TN	True Negative
TP	True Positive
ULB	Université Libre de Bruxelles
VC	Vapnik-Chervonenkis
XGBoost	Extreme Gradient Boosting

Contents

1	Introduction	12
1.1	Background	12
1.2	Motivation	12
1.3	Objectives	13
1.4	Organisation of the report	13
2	Literature Review	14
2.1	Introduction	14
2.2	Review of related work	14
2.3	Summary	16
3	Dataset and Methodology	17
3.1	The dataset	17
3.2	Data cleaning	18
3.3	Experimental protocol	18
3.4	Preventing leakage	19
3.5	Evaluation metrics	19
3.6	Software environment	20
4	Implementation	21
4.1	Unit I – Data exploration, validation and model complexity	21
4.2	Unit II – Feature selection and feature reduction	22
4.3	Unit II – Decision tree learning (ID3)	24
4.4	Unit II – Instance based learning: k-nearest neighbour	25
4.5	Unit III – Bayesian learning and Bayes optimal decisions	26
4.6	Unit III – Support vector machine	27
4.7	Unit IV – Artificial neural networks	28

4.8 Unit V – Ensemble learning	30
4.9 Unit V – Clustering	30
5 Results and Discussion	33
5.1 Overall comparison	33
5.2 The accuracy column	34
5.3 Errors in absolute terms	35
5.4 Cross-validated comparison	36
5.5 Discussion	38
6 Conclusion and Future Work	40
6.1 Conclusion	40
6.2 Limitations	41
6.3 Future work	41
References	42
Appendix A: Syllabus Coverage	44
Appendix B: Reproducing the Results	46
6.4 Repository	46
6.5 Steps	46
6.6 Output files	46

List of Figures

3.1	Class distribution in the cleaned dataset	18
4.1	Underfitting and overfitting as model complexity increases	22
4.2	The curse of dimensionality measured on the dataset	22
4.3	Training data projected onto the first two principal components . .	23
4.4	A single LDA discriminant component	23
4.5	Information gain of the best split per attribute	24
4.6	k-NN performance against the number of attributes	26
4.7	Naive Bayes posteriors and reliability diagram	27
4.8	Calibration brings the empirical optimum towards the theoretical threshold	28
4.9	Margin, decision boundary and support vectors on a two- dimensional projection	29
4.10	Perceptron decision boundaries for four logic gates	29
4.11	The perceptron cannot converge on XOR	30
4.12	Bagging and boosting compared against a single tree	31
4.13	Base learners, fixed rule fusion and trained rule fusion	31
4.14	Cluster sizes and fraud concentration per cluster	32
4.15	Dendrogram of credit card transactions (Ward linkage)	32
5.1	Precision, recall and F1-score for every model	35
5.2	Frauds caught and missed by each model	37
5.3	Cross-validated comparison on the training data	38

List of Tables

3.1	Dataset properties before and after cleaning	17
3.2	The train/test split	18
3.3	Evaluation metrics	20
3.4	Software environment	20
5.1	Performance of all twelve models on the test set	33
5.2	Frauds caught, frauds missed and false alarms	35
5.3	Five-fold cross-validated F1-score	36

Chapter 1

Introduction

1.1 Background

Credit card fraud occurs when a card or its details are used to make a transaction without the cardholder's authorisation. As payment volumes have moved online the problem has grown, and manual review of transactions has become impractical: a mid-sized issuer processes millions of transactions a day, of which a vanishing fraction are fraudulent.

Machine learning offers a route to automating this. A model trained on historical transactions labelled as fraudulent or legitimate can learn patterns that distinguish the two and apply them to new transactions in milliseconds.

1.2 Motivation

What makes this a genuinely difficult machine learning problem is not the detection itself but the **rarity** of the positive class. In the dataset used here, fraudulent transactions account for **0.1667%** of all records. Two consequences follow immediately, and both shape every decision made in this project.

First, **accuracy becomes meaningless**. A classifier that ignores its input and predicts "legitimate" for every transaction achieves 99.8333% accuracy while catching no fraud whatsoever. Any evaluation that leads with accuracy is therefore uninformative at best and misleading at worst.

Second, the **minority class is sparse**. Only 331 fraudulent transactions are

available in the training set to define the fraud region of a thirty-dimensional space. This limits how complex a model can usefully be, a point made precise by computational learning theory in Section 4.7.

1.3 Objectives

1. To apply the algorithms taught across all five units of the Machine Learning syllabus to a single real classification problem.
2. To establish a single leakage-free experimental protocol so that every model is trained and evaluated on identical data and the comparison is valid.
3. To evaluate twelve models using metrics appropriate to extreme class imbalance, and to identify which performs best.
4. To verify experimentally several theoretical results from the syllabus – the perceptron convergence theorem, the curse of dimensionality, the Bayes optimal decision rule, and the bias-variance behaviour of ensembles.
5. To produce a fully reproducible implementation, seeded with the last two digits of the author’s roll number as required.

1.4 Organisation of the report

Chapter 2 reviews related work. Chapter 3 describes the dataset and the experimental methodology. Chapter 4 presents the implementation of each algorithm, organised by syllabus unit. Chapter 5 presents and discusses the results. Chapter 6 concludes and identifies future work.

Chapter 2

Literature Review

2.1 Introduction

Credit card fraud detection has been studied extensively as a supervised classification problem. This chapter reviews five representative studies and identifies the methodological points that informed the design of this project.

2.2 Review of related work

Awoyemi, Adetunmbi and Oluwadare (2017) compared naive Bayes, k-nearest neighbour and logistic regression on the same ULB dataset used here. They applied a hybrid of undersampling the majority class and oversampling the minority class to create artificial 10:90 and 34:66 distributions, reporting optimal accuracies of 97.92%, 97.69% and 54.86% respectively and concluding that k-NN performed best [1].

Two methodological observations follow. Because the classifiers were evaluated on artificially rebalanced distributions, the figures describe performance on data that does not occur in production, where the fraud rate is 0.17% rather than 10% or 34%. Further, the resampling was applied before the train/test division, so information about test instances influenced the training distribution. The present project deliberately avoids both, as described in Section 3.4.

Varmedja et al. (2019) applied SMOTE oversampling and compared logistic regression, Random Forest, naive Bayes and a multilayer perceptron, reporting

high accuracy across all four. They qualify their own result candidly: *“accuracy is extremely high, although that doesn’t mean that results are perfect. Accuracy must be taken ‘with a grain of salt’ – desirably it should be interpreted in combination with some other metrics”* [5]. This project takes that warning as a starting point rather than a caveat.

Tiwari et al. (2021) survey nine technique families, from Hidden Markov Models to convolutional networks. Their conclusion notes two limitations of the field: that no reviewed technique detects fraud at the moment it occurs, and that existing techniques *“give accurate results only when performed on a particular dataset and sometimes with some special features only”* [4].

Hernandez Aros et al. (2024) conducted a systematic review of 104 articles published between 2012 and 2023. Their survey of method usage is directly relevant to the choice of algorithms here: supervised learning accounts for 56.73% of studies, and Random Forest is the most frequently applied model (34 studies), followed by logistic regression (32), decision trees and SVM (29 each), naive Bayes (19), artificial neural networks (17) and k-NN (14) [3]. The algorithms selected for this project therefore coincide closely with those most used in the literature – which is unsurprising, since both are drawn from the standard machine learning curriculum.

The same review criticises the field’s reliance on ageing datasets, observing that studies use data that *“because of their age, do not provide effective and accurate results in the current context as a result of the new fraud modalities created day after day”* [3]. This is a fair criticism of the benchmark used here and is acknowledged among the limitations in Section 6.2.

Compagnino et al. (2025) evaluated tree ensembles on proprietary banking data using a careful protocol including temporal splits and customer-level grouping. Their best Random Forest achieved precision 0.93 but recall 0.34, which they describe as *“an unacceptable level of risk for a financial institution”* [2]. They further observe that accuracy and weighted F1 around 0.97 *“are misleading in the presence of extreme class imbalance and mask the model’s critical failure to identify the minority class”*. Their results establish that high headline metrics on this problem are compatible with poor operational performance, reinforcing the metric choices made in Section 3.5.

2.3 Summary

Three conclusions were drawn from the review and applied to this project: accuracy must not be used as the headline metric; resampling must never be applied before the train/test split; and the algorithms chosen – Random Forest, decision trees, SVM, naive Bayes, ANN and k-NN – are precisely those the literature uses most, so the comparison is a meaningful one.

Chapter 3

Dataset and Methodology

3.1 The dataset

The dataset was published by the Machine Learning Group of the Université Libre de Bruxelles and is distributed through Kaggle [6]. It records transactions made by European cardholders over two days in September 2013.

Table 3.1: Dataset properties before and after cleaning

Property	Raw	After cleaning
Transactions	284,807	283,726
Attributes	30 + label	30 + label
Fraudulent	492	473
Fraud rate	0.1727%	0.1667%
Duplicate rows	1,081	0
Missing values	0	0
Time span	172,792 s (~48 h)	172,792 s
Mean fraudulent amount	EUR 122.21	EUR 123.87
Mean legitimate amount	EUR 88.29	EUR 88.41

A property of the dataset deserves particular attention. Attributes V1 to V28 are **not** the original transaction attributes: they are **principal components** obtained by applying PCA to the original features before publication, in order

to protect cardholder confidentiality. Only Time, Amount and Class are untransformed. PCA is a Unit II topic, and the fact that the benchmark itself is the output of that technique is examined in Section 4.2.

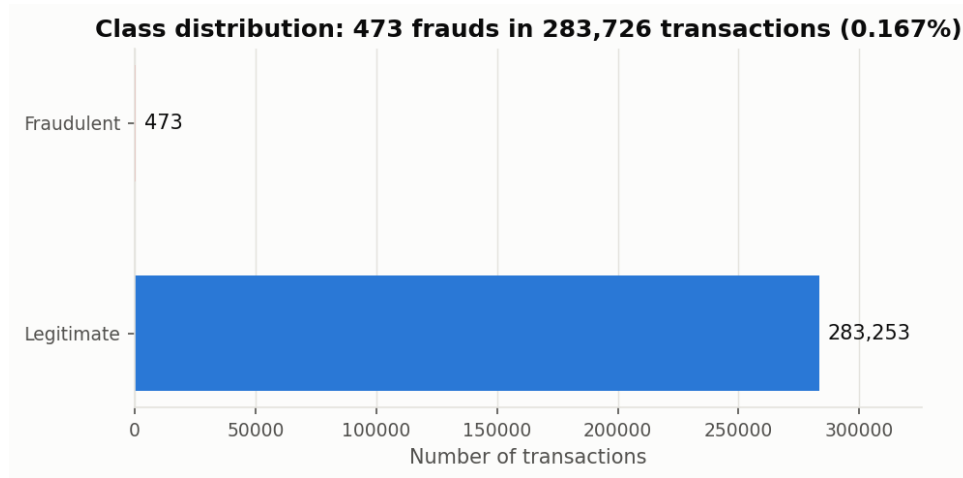


Figure 3.1: Class distribution in the cleaned dataset

3.2 Data cleaning

The raw file contains **1,081 exact duplicate rows**, of which 19 are fraudulent. These were removed before any other processing. The reason is a subtle form of leakage: if a transaction appears twice and the split places one copy in the training set and the other in the test set, the model is evaluated on a record it has already memorised, and the reported performance is optimistic.

3.3 Experimental protocol

A single **stratified** train/test split was performed with `test_size = 0.30` and `random_state = 77`, and every model in this project uses that same split.

Table 3.2: The train/test split

	Transactions	Frauds	Fraud rate
Training set	198,608	331	0.1667%
Test set	85,118	142	0.1668%

Stratification preserves the class proportion in both partitions, which matters when the positive class is 0.17% of the data: an unstratified random split could easily produce partitions with substantially different fraud rates.

The seed 77 is the last two digits of the author’s roll number AP24110010677, as required. It is applied to the split, to every model with internal randomness, and to every subsample, so that all results are exactly reproducible.

For model selection within the training data, **5-fold stratified cross validation** was used. The training set is divided into five parts; the model is trained on four and validated on the fifth, rotating through all five. This gives a more stable estimate than a single validation split, which is valuable when the positive class is small.

3.4 Preventing leakage

Three rules were enforced throughout:

1. **No resampling before the split.** Where rebalancing was applied, it occurred inside cross-validation folds on training data only. This differs from the approach of Awoyemi et al. [1] discussed in Section 2.2.
2. **No scaling statistic computed on test data.** Standardisation is a step inside each model’s Pipeline object, so means and variances are fitted on training folds alone.
3. **Duplicates removed before splitting**, as described in Section 3.2.

Two models – k-nearest neighbour and the RBF support vector machine – cannot be fitted on 198,608 rows within the available compute, since k-NN prediction costs $O(n_{train} \times n_{test})$ and SVM training scales worse than quadratically. For these two models only, the **training set** was subsampled while retaining every fraudulent case. The test set was never touched, so the comparison remains valid; the subsampling is declared in the results table.

3.5 Evaluation metrics

Given the 0.1667% prevalence, accuracy is reported only once in the results, next to the 99.8333% achieved by the trivial classifier, to demonstrate its inadequacy. The metrics used for evaluation are:

Table 3.3: Evaluation metrics

Metric	Definition	Why it is used
Precision	$TP/(TP + FP)$	Of the transactions flagged, how many are really fraud
Recall	$TP/(TP + FN)$	Of the frauds present, how many are caught
F1-score	Harmonic mean of the two	Single balanced summary of the fraud class
PR-AUC	Area under the precision-recall curve	Threshold-independent; appropriate at low prevalence
ROC-AUC	Area under the ROC curve	Reported for comparability with prior work
Confusion matrix	TP, FP, TN, FN counts	Errors in absolute, interpretable terms

3.6 Software environment

Table 3.4: Software environment

Component	Version	Role
Python	3.11	Runtime
scikit-learn	1.8.0	Models, pipelines, metrics, cross validation
XGBoost	3.2.0	Gradient boosted trees
pandas	3.0.2	Data handling
NumPy	2.4.4	Numerical computation
SciPy	1.11+	Hierarchical clustering linkage
Matplotlib	3.8+	Figures

Chapter 4

Implementation

The implementation is organised as ten Jupyter notebooks, each covering one group of syllabus topics. A shared module `utils.py` provides the data loading, the single train/test split and the evaluation functions, so that every notebook uses identical data and identical metric definitions.

4.1 Unit I - Data exploration, validation and model complexity

Notebook 01 establishes the properties of the dataset and the experimental protocol described in Chapter 3. Two demonstrations are worth reporting here.

Why accuracy fails. The trivial “always legitimate” classifier was constructed and evaluated: 99.8333% accuracy, 0 of 142 frauds detected. Every accuracy figure in Chapter 5 must be read against this number.

Overfitting and underfitting. A decision tree was trained at increasing maximum depths and its F1-score measured on both the training and the test set.

At small depths both curves are low: the tree is too simple and **underfits**. As depth increases the training F1 climbs towards 1.0 while the test F1 flattens and declines – the tree is **overfitting**, memorising the training set. The best test performance occurs at moderate depth, which is why the tree used in Section 4.3 is pruned rather than grown without limit.

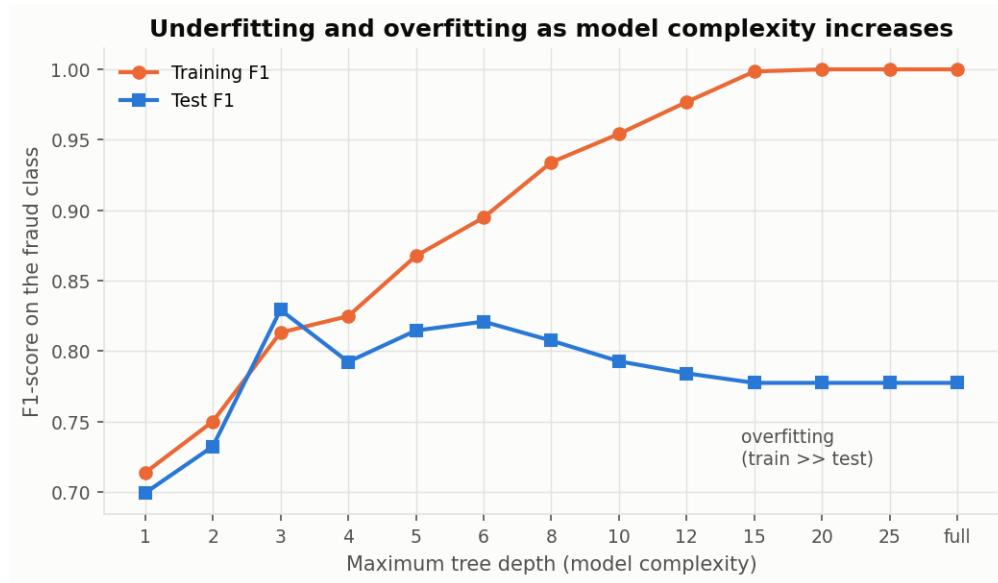


Figure 4.1: Underfitting and overfitting as model complexity increases

4.2 Unit II - Feature selection and feature reduction

Notebook 02 addresses dimensionality. The **curse of dimensionality** was measured directly on the data by computing, for samples of increasing dimension, the mean ratio between each point's nearest and farthest neighbour distance.

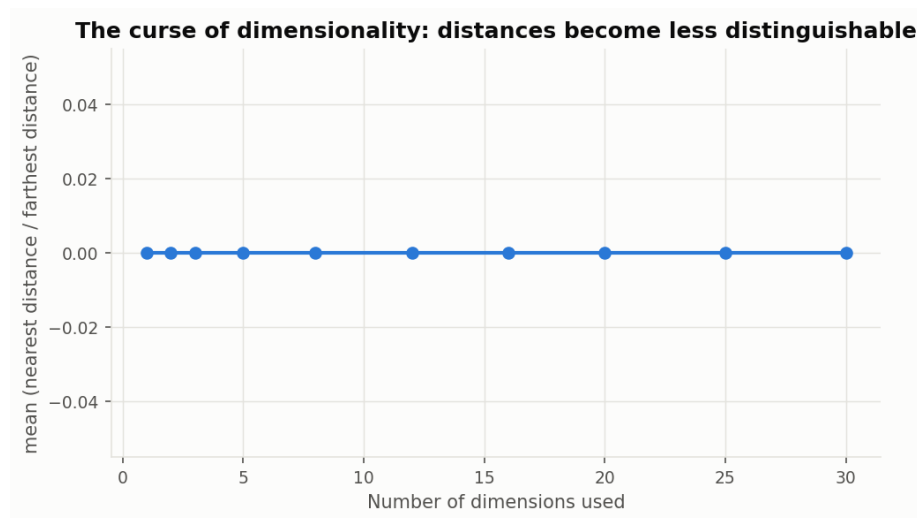


Figure 4.2: The curse of dimensionality measured on the dataset

The ratio rises steadily with dimension: in one dimension the nearest point is far closer than the farthest, but by thirty dimensions the two are much more similar, so “nearest” carries less information. This predicts difficulty for k-nearest

neighbour, a prediction tested in Section 4.4.

Three families of feature selection were applied. **Filter** methods score each attribute independently (ANOVA F-test and mutual information); the **wrapper** method Recursive Feature Elimination evaluates attributes jointly by retraining a model as attributes are removed; **embedded** selection uses the importances a tree assigns during training. All three rankings agree closely, placing V17, V14, V12, V10 and V4 near the top.

For feature reduction, **PCA** and **LDA** were compared. PCA is unsupervised and maximises variance; LDA is supervised and maximises class separation. The contrast is visible:

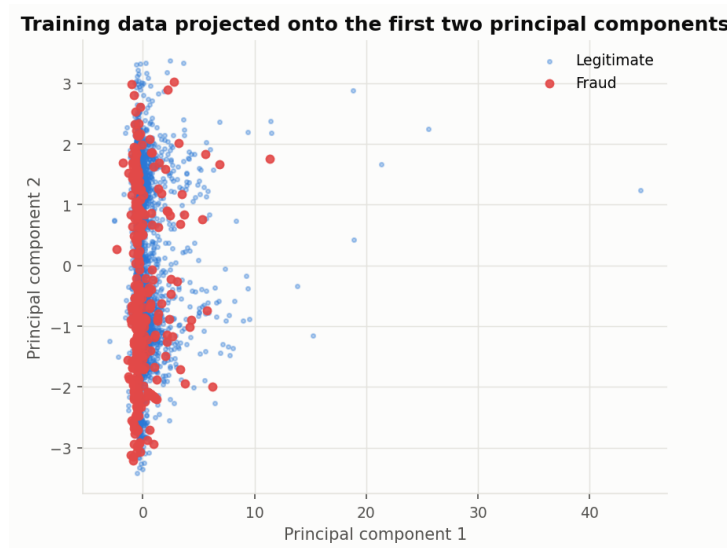


Figure 4.3: Training data projected onto the first two principal components

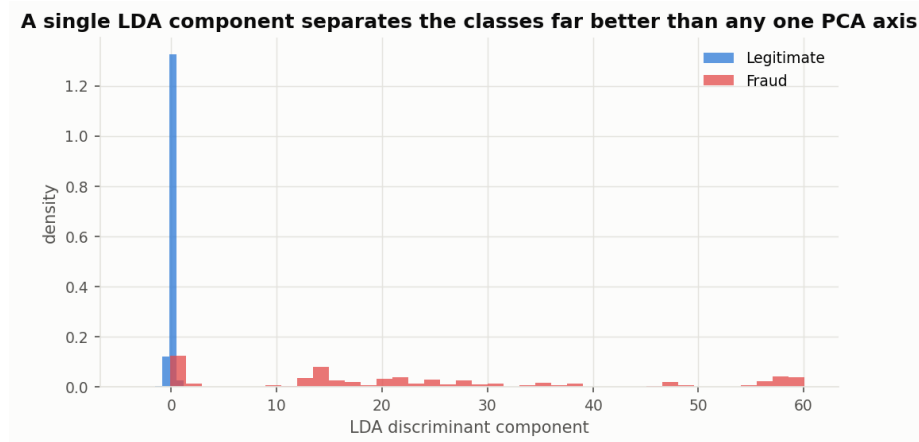


Figure 4.4: A single LDA discriminant component

Two PCA components leave the classes heavily overlapped; a single LDA component separates them into visibly distinct distributions. This is exactly what the two objectives predict - PCA was never attempting to separate the classes.

A note on applying PCA here: because V1-V28 are already principal components, they are already mutually uncorrelated, so a second PCA mainly reorders and rescales them, and most components are needed to retain 95% of the variance.

4.3 Unit II - Decision tree learning (ID3)

Notebook 03 implements decision tree learning. **Entropy** and **information gain** were computed by hand from their definitions:

$$Entropy(S) = - \sum_i p_i \log_2 p_i, \quad Gain(S, A) = Entropy(S) - \sum_v \frac{|S_v|}{|S|} Entropy(S_v)$$

The entropy of the root node is only **0.0192 bits**, rather than the 1.0 bit of a balanced two-class problem - a direct consequence of the imbalance, since the set is already almost pure. The best split per attribute was then searched exactly as ID3 would, and the resulting ranking reproduced the attribute the library implementation selects.

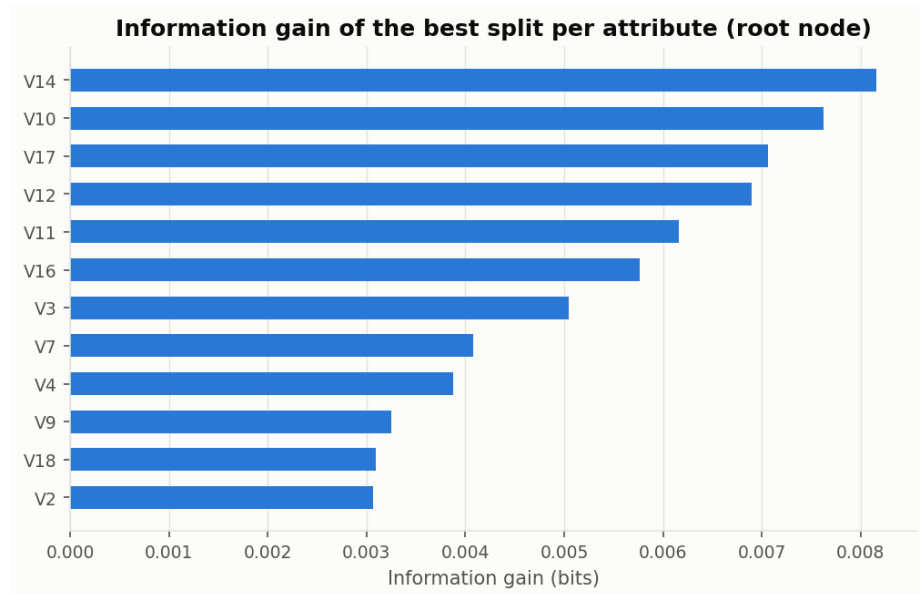


Figure 4.5: Information gain of the best split per attribute

The **hypothesis space** of ID3 is complete – any discrete function can be represented by some tree – so its **inductive bias** is not a restriction on what it can express but a **preference** arising from the order of search: shorter trees, with high-information-gain attributes near the root, are preferred. This is a preference bias rather than a restriction bias, and a form of Occam’s razor.

The pruning depth was chosen by cross validation. A note on class weighting: the fraud class was initially re-weighted by the inverse of its frequency (approximately 600:1). On a pruned tree this proved counter-productive – the leaves are impure by design, and the weighted vote tipped a very large number of them to “fraud”, producing over three thousand false alarms. The tree reported in Chapter 5 is therefore unweighted, and the imbalance is handled through the choice of metric and through the threshold analysis of Section 4.5.

4.4 Unit II - Instance based learning: k-nearest neighbour

Notebook 04 implements k-NN. A complete **numerical problem** was worked step by step: Euclidean distances from a query transaction to each stored example, sorting, selection of the k nearest, and the majority vote. The hand calculation reproduced scikit-learn’s neighbours, distances and prediction exactly.

Standardisation is mandatory before computing distances, because Amount ranges to approximately 25,000 while the principal components lie within roughly ± 5 ; without scaling the distance would be dominated by Amount alone.

The value of k was chosen by cross validation rather than left at 1, which would overfit. The effect of dimensionality was then measured by training k-NN on progressively more attributes:

Performance **peaks at a modest number of well-chosen attributes and then declines** as more are added, even though the additional attributes contain genuine information. This confirms the curse of dimensionality on real data and provides a concrete justification for the feature selection work of Section 4.2.

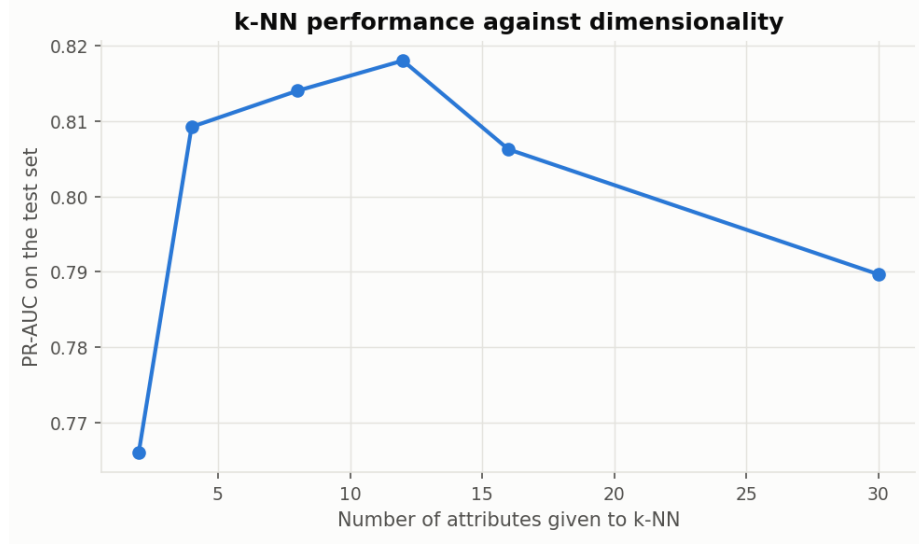


Figure 4.6: k-NN performance against the number of attributes

4.5 Unit III - Bayesian learning and Bayes optimal decisions

Notebook 05 implements naive Bayes. The classifier applies Bayes' theorem

$$P(C_k | x_1, \dots, x_n) \propto P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

under the assumption that attributes are conditionally independent given the class. That assumption is normally the method's weak point, but here it holds unusually well: the mean absolute correlation between attributes is very low, because V1-V28 are principal components and therefore constructed to be uncorrelated. A Unit II property of the data strengthens a Unit III method.

The more substantial part of this notebook concerns the **decision rule**. The default predict applies the **MAP rule**, classifying as fraud whenever $P(\text{fraud} | x) > 0.5$. That rule is optimal only when both kinds of error cost the same, which is not the case in fraud detection. The **Bayes optimal decision** minimises expected loss instead. Writing C_{FN} for the cost of a missed fraud, C_{FP} for the cost of a false alarm and $p = P(\text{fraud} | x)$:

$$\mathbb{E}[\text{loss} | \text{flag}] = (1 - p) C_{FP}, \quad \mathbb{E}[\text{loss} | \text{allow}] = p C_{FN}$$

Flagging is optimal when $(1 - p) C_{FP} < p C_{FN}$, which rearranges to

$$p > \frac{C_{FP}}{C_{FP} + C_{FN}} = \tau^*$$

With C_{FN} anchored to the mean fraudulent amount (EUR 123.87) and C_{FP} taken as one fiftieth of it, $\tau^* = 1/51 \approx 0.0196$ - about twenty-six times below the default threshold of 0.5.

Applying this threshold produced an unexpected result: the empirical minimum of the loss curve lay several orders of magnitude below τ^* . The cause was identified as **poor calibration**. The derivation assumes p is a genuine probability, but multiplying thirty likelihood terms drives naive Bayes posteriors towards 0 and 1, so the scores rank well without being probabilities.

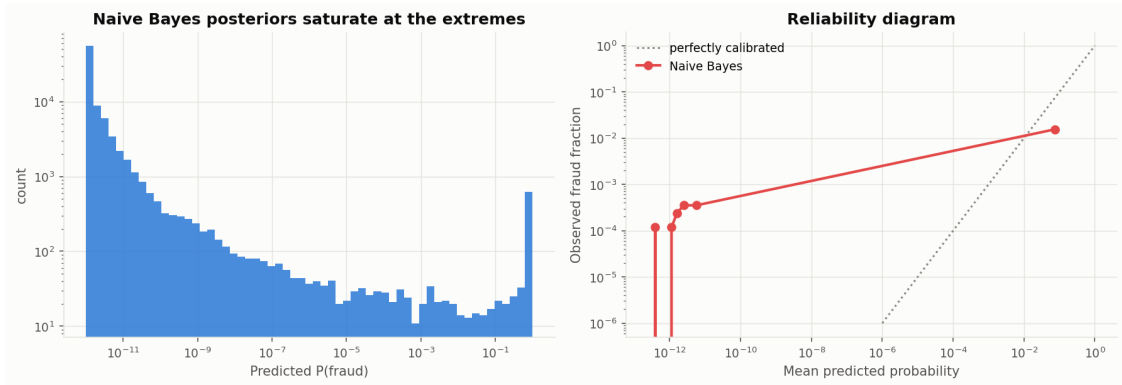


Figure 4.7: Naive Bayes posteriors and reliability diagram

Applying isotonic calibration moved the empirical optimum substantially closer to the theoretical value, completing the argument:

The conclusion is that the Bayes optimal rule is correct, but can only be **applied** to a model whose outputs are genuine posterior probabilities. This is a more useful result than the derivation alone, and it could not have been reached without running the experiment.

4.6 Unit III - Support vector machine

Notebook 06 implements the SVM. The primal problem maximises the margin by minimising $\frac{1}{2}\|w\|^2$ subject to $y_i(w^T x_i + b) \geq 1$. The **dual formulation**

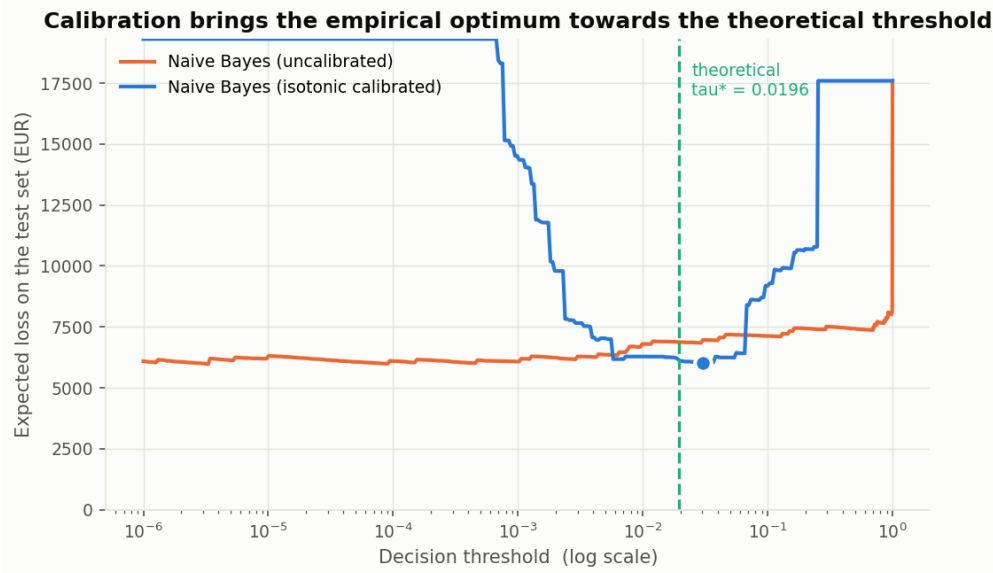


Figure 4.8: Calibration brings the empirical optimum towards the theoretical threshold

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

expresses the problem purely in terms of inner products between training points. This has two consequences: the solution is sparse, depending only on the **support vectors**, and the inner product can be replaced by a **kernel**, giving a non-linear boundary without ever computing the high-dimensional mapping.

Soft-margin slack variables permit controlled violations, with the parameter C trading margin width against training error. Linear, polynomial and RBF kernels were compared and RBF performed best.

The figure makes the theory visible: the solid line is the decision boundary, the dashed lines mark the margin, and the circled points are the support vectors. The linear kernel produces a straight boundary; the RBF kernel curves around the fraud cluster.

4.7 Unit IV - Artificial neural networks

Notebook 07 works through the historical development of neural networks as the syllabus presents it.

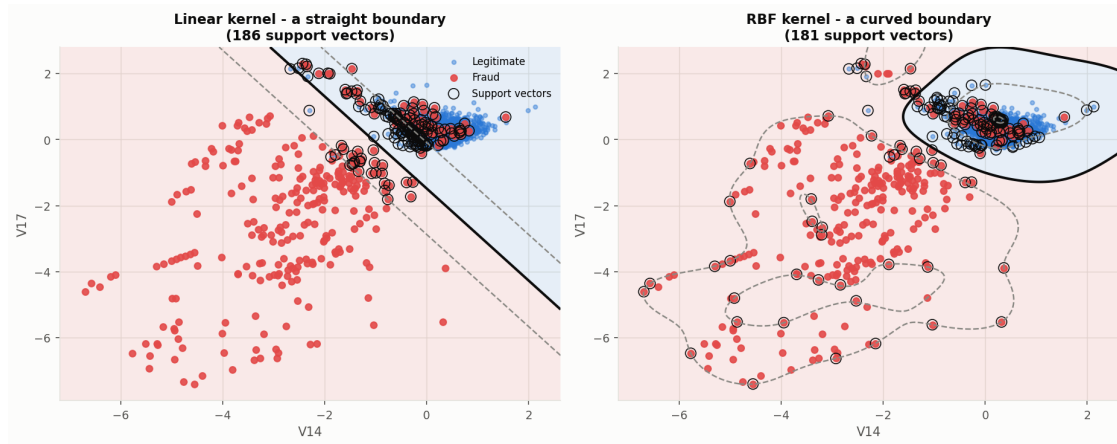


Figure 4.9: Margin, decision boundary and support vectors on a two-dimensional projection

A **McCulloch-Pitts neuron** was constructed with hand-set weights to compute AND and OR. It has no learning rule. The **perceptron** adds one, updating weights by $w_i \leftarrow w_i + \eta(t - o)x_i$, and learned AND, OR, NAND and NOR from examples within a few epochs - each being linearly separable, as the perceptron convergence theorem requires.

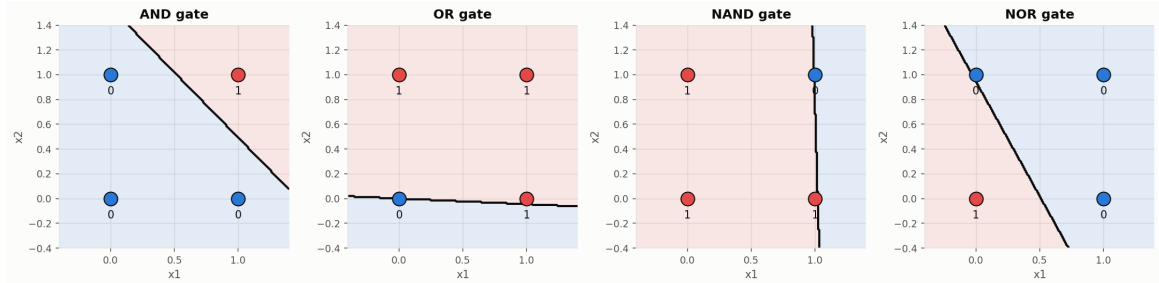


Figure 4.10: Perceptron decision boundaries for four logic gates

It then **failed on XOR**, and its error count never reached zero:

This is the limitation Minsky and Papert identified in 1969. **Gradient descent** replaces the hard threshold with a differentiable objective, enabling multiple layers, and the learning rate governs whether the descent converges smoothly, slowly, or oscillates. A **multilayer network trained by backpropagation** then solved XOR, producing a boundary that is no longer a straight line.

Finally, **computational learning theory** was applied. The VC dimension of a perceptron in d dimensions is $d + 1 = 31$; the network's is in the low thousands. The PAC sample-complexity bound is loose, but its message holds: the

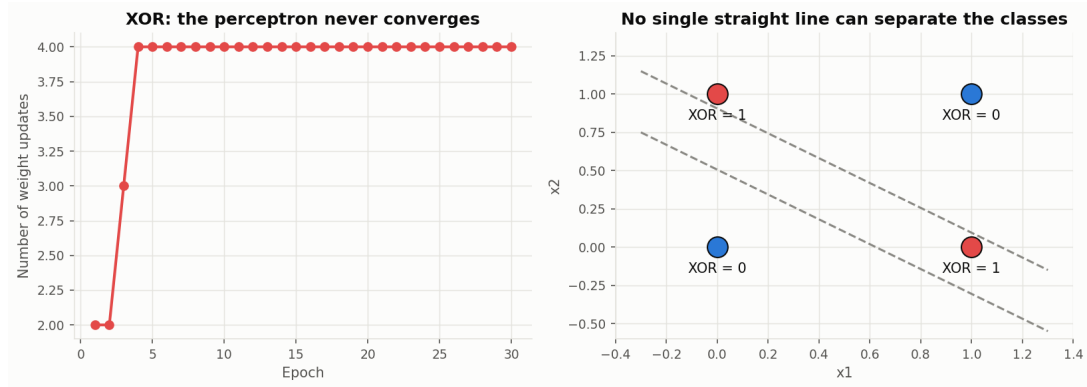


Figure 4.11: The perceptron cannot converge on XOR

project has roughly 200,000 training examples in total, yet only **331 of them are fraudulent**, and the minority class is where the boundary must be learned. This justifies the regularisation, early stopping and small architecture used, and explains why high-capacity models do not dominate the results.

4.8 Unit V - Ensemble learning

Notebook 08 implements ensembles. **Bagging** trains base learners in parallel on bootstrap samples and reduces **variance**; **Random Forest** adds attribute subsampling at each split, which decorrelates the trees and makes averaging more effective. **Boosting** trains learners sequentially, each concentrating on the previous errors, and reduces **bias**; AdaBoost reweights examples while XGBoost fits each tree to the residuals.

Rule fusion combines the outputs of *different* model types. A **fixed rule** is decided in advance and learns nothing: majority vote, mean, product, max and min rules were all implemented from their definitions and compared. A **trained rule** learns the combination instead – stacking was implemented with out-of-fold predictions feeding a logistic regression meta-learner, whose coefficients show explicitly how much each base model is trusted.

4.9 Unit V - Clustering

Notebook 09 applies unsupervised learning. **K-means** minimises the within-cluster sum of squares by alternating assignment and centroid update; k was

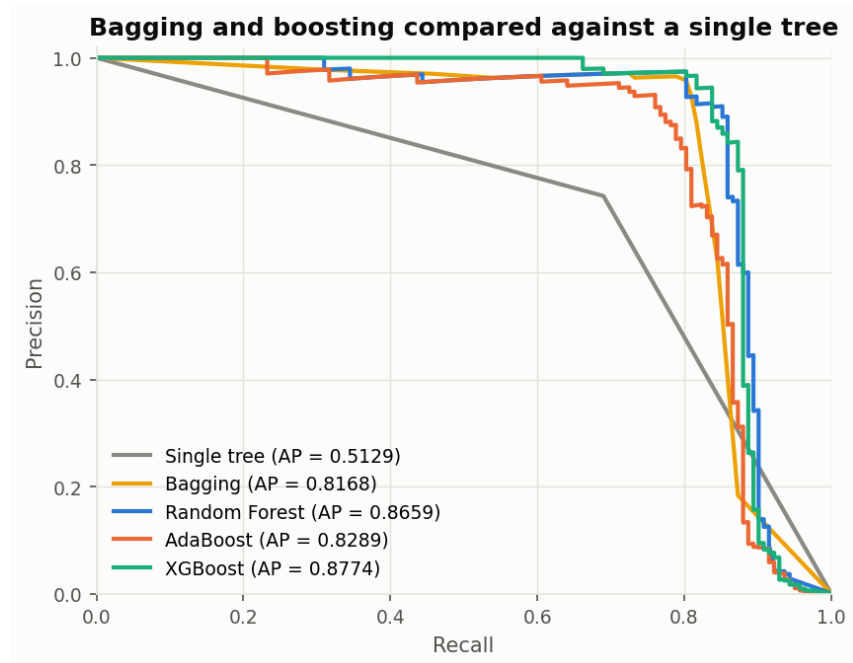


Figure 4.12: Bagging and boosting compared against a single tree

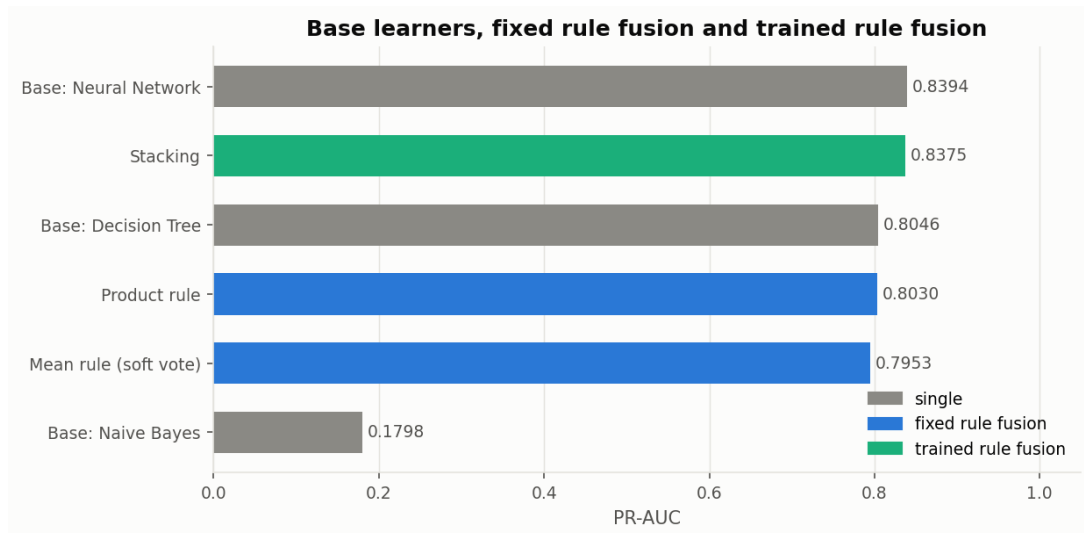


Figure 4.13: Base learners, fixed rule fusion and trained rule fusion

chosen using the elbow method and the silhouette score. The labels were withheld entirely during clustering and revealed only afterwards to inspect the result.

The clusters are far from uniform in fraud content: at least one contains fraud at many times the overall rate. K-means recovered part of the fraud structure **without ever being shown a label**.

Distance to the nearest centroid also works as an unsupervised anomaly score,

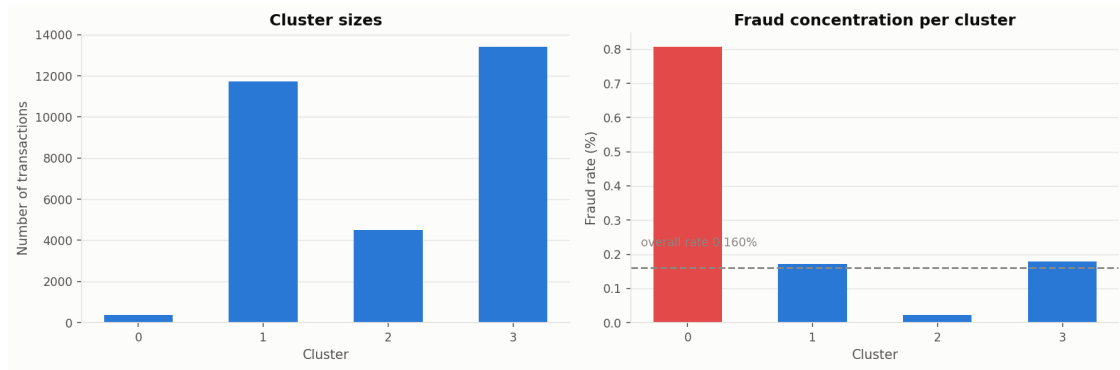


Figure 4.14: Cluster sizes and fraud concentration per cluster

and fraudulent transactions sit measurably further from every centroid than legitimate ones.

Hierarchical clustering builds a dendrogram rather than a flat partition, so k need not be fixed in advance. Its $O(n^2)$ memory cost confined it to a 2,000-row sample – a real limitation of the method. Single linkage exhibited the expected chaining effect, producing one dominant cluster; Ward linkage gave the most balanced partition.

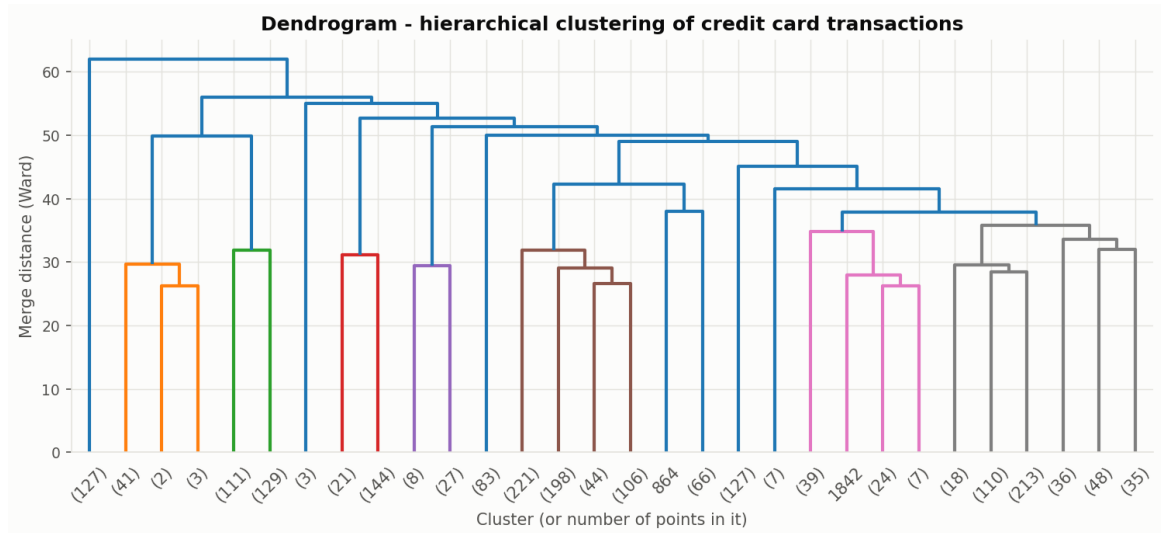


Figure 4.15: Dendrogram of credit card transactions (Ward linkage)

Chapter 5

Results and Discussion

All results were produced by the code described in Chapter 4 and evaluated on the same held-out test set of **85,118 transactions containing 142 frauds**.

5.1 Overall comparison

Table 5.1: Performance of all twelve models on the test set

Model	Unit	Accuracy	Precision	Recall	F1	PR-AUC
XGBoost	V	0.9996	0.9508	0.8169	0.8788	0.8774
AdaBoost	V	0.9995	0.9231	0.7606	0.8340	0.8289
Random Forest	V	0.9995	0.9717	0.7254	0.8306	0.8659
Decision Tree	II	0.9994	0.8672	0.7817	0.8222	0.8075
Neural Network (MLP)	IV	0.9994	0.9204	0.7324	0.8157	0.8498
Voting (fixed rule fusion)	V	0.9994	0.8833	0.7465	0.8092	–

Model	Unit	Accuracy	Precision	Recall	F1	PR-AUC
k-Nearest Neighbour	II	0.9994	0.8438	0.7606	0.8000	0.7897
Support Vector Machine	III	0.9993	0.8333	0.7394	0.7836	0.7841
Stacking (trained rule fusion)	V	0.9993	0.9474	0.6338	0.7595	0.8375
Perceptron	IV	0.9991	0.6761	0.8380	0.7484	–
Naive Bayes	III	0.9895	0.1128	0.7746	0.1970	0.1828
K-Means (unsupervised)	V	0.9904	0.1045	0.6268	0.1791	0.1572

5.2 The accuracy column

The accuracy column deserves separate comment. Every model scores between 0.9895 and 0.9996 – a spread of one percentage point – and the trivial “always legitimate” classifier scores **0.998333**. Several models differ from it by less than **0.05 percentage points**, while their recall ranges from 0.6268 to 0.8380.

A reader given only the accuracy column would have no basis for distinguishing these models, and would conclude that a classifier detecting nothing is competitive with the best. This is the concrete demonstration of the point made in Section 1.2, and the reason every other metric in this report is reported on the fraud class specifically.

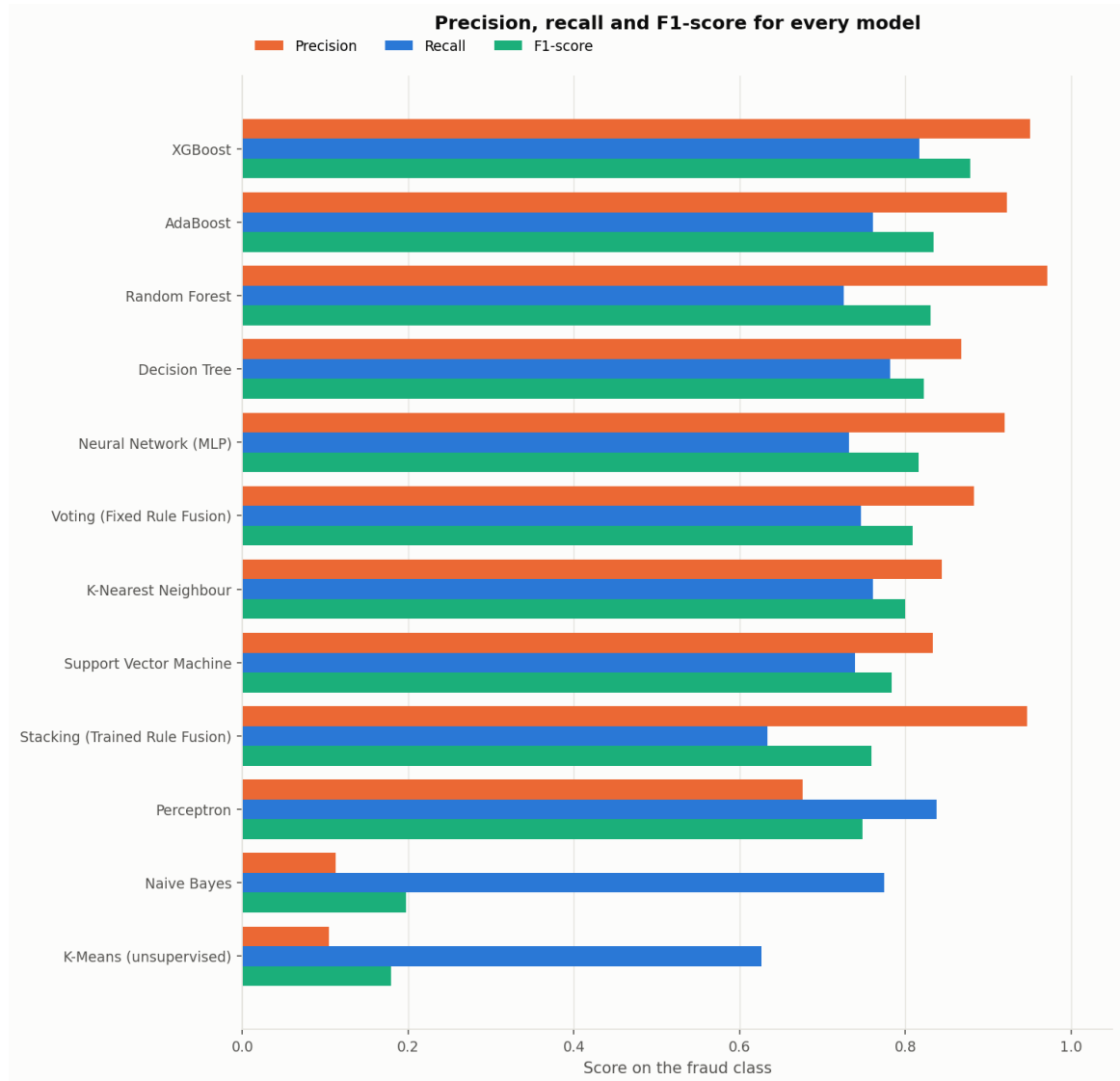


Figure 5.1: Precision, recall and F1-score for every model

5.3 Errors in absolute terms

Percentages obscure operational meaning. The test set contains 142 fraudulent transactions.

Table 5.2: Frauds caught, frauds missed and false alarms

Model	Frauds caught (of 142)	Frauds missed	False alarms
Perceptron	119	23	57
XGBoost	116	26	6

Model	Frauds caught (of 142)	Frauds missed	False alarms
Naive Bayes	110	32	865
AdaBoost	108	34	9
k-Nearest Neighbour	108	34	20
Voting (fixed rule fusion)	106	36	14
Support Vector Machine	105	37	21
Neural Network (MLP)	104	38	9
Random Forest	103	39	3
Decision Tree	111	31	17
Stacking (trained rule fusion)	90	52	5
K-Means (unsupervised)	89	53	763

The table reveals a trade-off invisible in the F1 column. The perceptron catches the most frauds (119) but raises 57 false alarms. Random Forest raises only **3** false alarms but misses 39 frauds. XGBoost achieves the best balance: 116 frauds caught for 6 false alarms.

Naive Bayes and K-means catch a reasonable number of frauds but raise hundreds of false alarms, which is why their F1-scores are so low. For naive Bayes this is partly deliberate – it is evaluated at the Bayes optimal threshold of Section 4.5, which is far below 0.5 and trades precision for recall by design.

5.4 Cross-validated comparison

To confirm the ranking is not an artefact of a single split, five models were re-evaluated with 5-fold stratified cross validation on the training data.

Table 5.3: Five-fold cross-validated F1-score

Model	CV F1 (mean)	Std. dev.	Training time (s)
Random Forest	0.8319	0.0127	59.0
XGBoost	0.8293	0.0304	9.0
Decision Tree	0.8011	0.0409	22.3
Perceptron	0.5843	0.2941	1.5
Naive Bayes	0.2381	0.0167	0.7

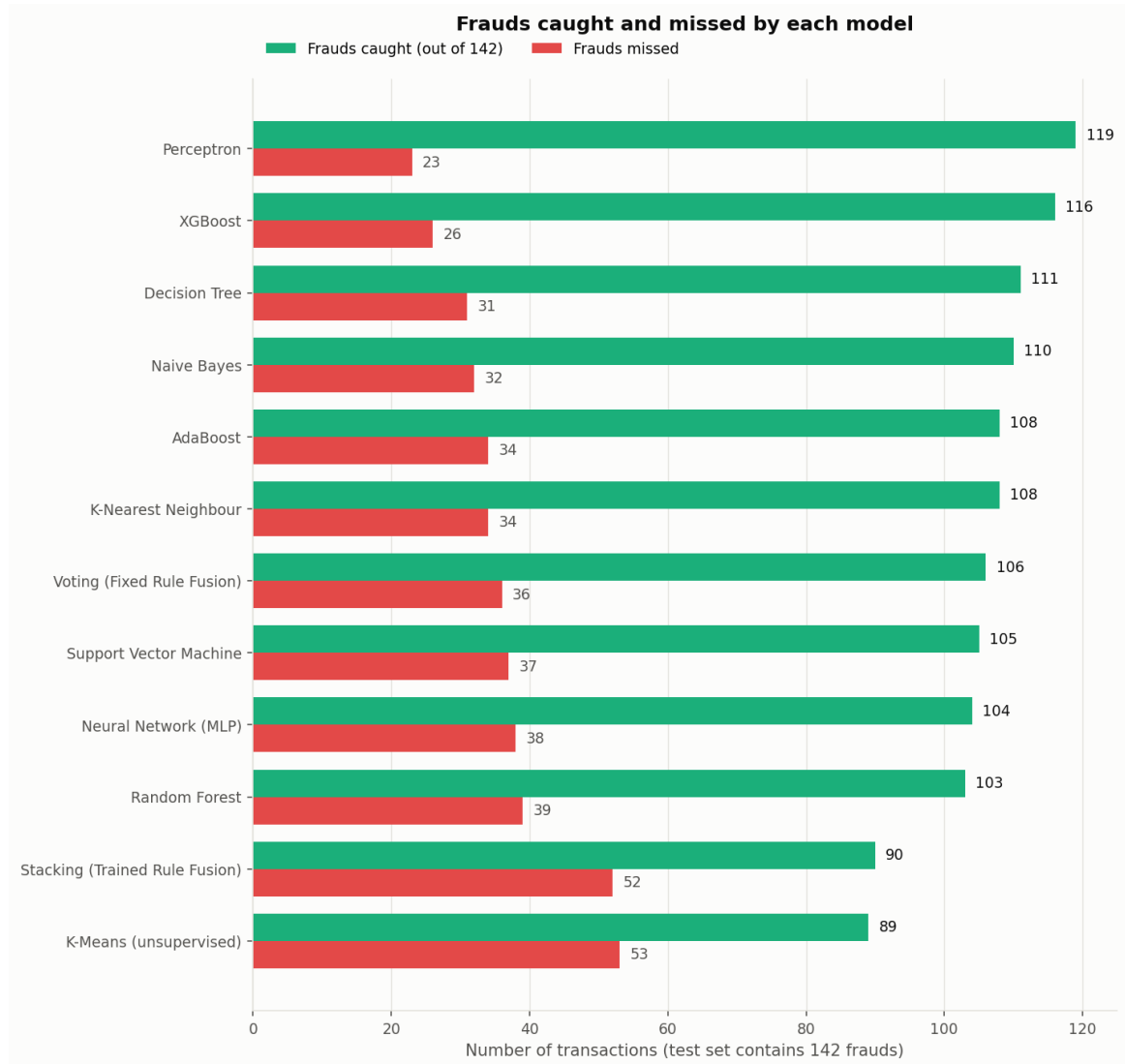


Figure 5.2: Frauds caught and missed by each model

The cross-validated ranking agrees with the held-out ranking, so the conclusion is a property of the models rather than of one fortunate split. The standard deviations are informative in their own right: the perceptron's 0.2941 is by far the largest, reflecting its instability on a non-separable problem, while Random Forest's 0.0127 reflects precisely the variance reduction that bagging is designed to deliver.

The training-time column deserves attention alongside the scores. XGBoost achieves essentially the same cross-validated F1 as Random Forest in **one sixth of the training time**, and naive Bayes trains in under a second – a difference of nearly two orders of magnitude for a model that is still catching 110 of 142

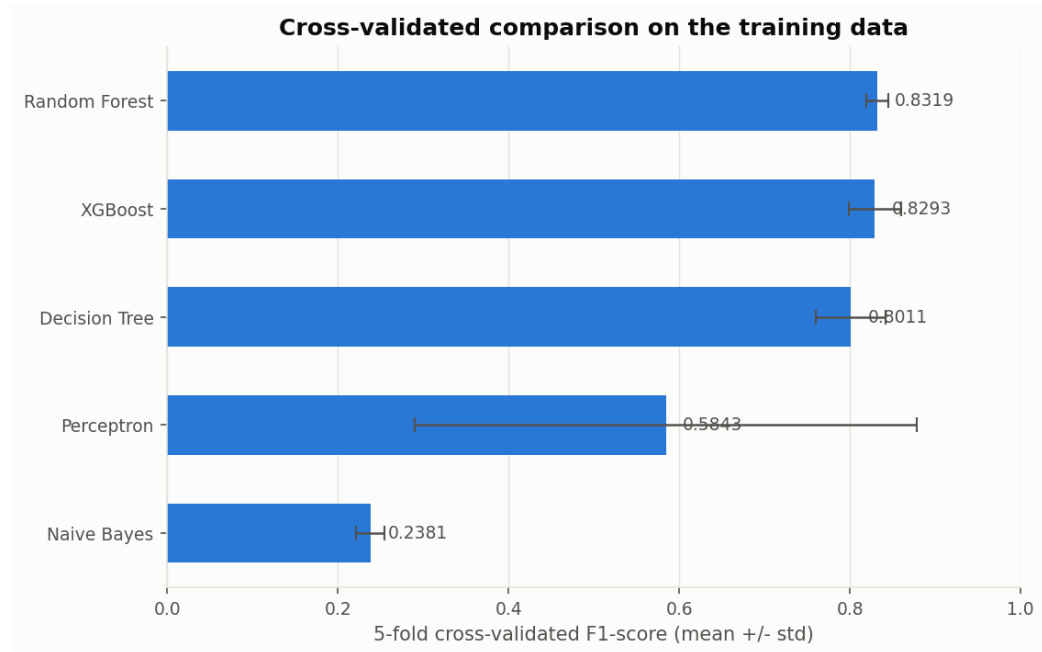


Figure 5.3: Cross-validated comparison on the training data

frauds.

5.5 Discussion

Ensemble methods perform best. The three highest F1-scores all belong to Unit V methods. Combining many learners reduced variance (Random Forest) and bias (AdaBoost, XGBoost) in a way no single model matched. Boosting outperformed bagging here, which is consistent with the bias-variance analysis: the decision boundary between fraud and legitimate traffic is complex, so reducing bias helps more than reducing variance.

Fixed rule fusion outperformed trained rule fusion. The voting ensemble scored 0.8092 against stacking's 0.7595. This is initially surprising, but explainable: the stacking meta-learner must itself be fitted, and with only 331 fraud examples available the fusion step has very little signal to learn from. A trained rule is more expressive than a fixed rule but needs data to justify that expressiveness – a small-sample version of the same argument the VC dimension analysis makes in Section 4.7.

Simple models are closer than expected, and far cheaper. The gap between the best ensemble (0.8788) and a single pruned decision tree (0.8222) is

smaller than the gap in training cost between them. Naive Bayes trains in under a second.

Theory predicted what the experiments found. Four theoretical results from the syllabus were confirmed empirically: the perceptron convergence theorem (convergence on four linearly separable gates, failure on XOR), the curse of dimensionality (k-NN performance declining as attributes were added), the Bayes optimal decision rule (with the calibration caveat of Section 4.5), and the variance-reduction property of bagging (visible in Random Forest's low cross-validation standard deviation).

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project applied the complete Machine Learning syllabus – all five units – to the detection of fraudulent credit card transactions, implementing twelve models under a single leakage-free experimental protocol with random seed 77.

XGBoost performed best, achieving precision 0.9508, recall 0.8169 and an F1-score of 0.8788, detecting 116 of 142 fraudulent transactions in the test set while raising only 6 false alarms. Five-fold cross validation confirmed the ranking, with Random Forest and XGBoost close together at the top.

The project’s central methodological finding is that **accuracy is unusable on this problem**. Every model scores above 98.9%, and most sit within a fraction of a percentage point of a classifier that detects no fraud at all. Model selection must be based on recall, precision, F1 and PR-AUC.

Beyond the comparison itself, four theoretical results from the syllabus were verified experimentally rather than merely stated: the perceptron converged on four linearly separable logic gates and failed on XOR; k-nearest neighbour performance declined as attributes were added, demonstrating the curse of dimensionality on real data; the Bayes optimal decision threshold derived from an asymmetric loss matrix could only be applied after calibrating the posteriors; and bagging produced exactly the variance reduction the bias-variance decomposition predicts.

6.2 Limitations

The dataset is dated and short. It covers two days of transactions from 2013 and is not representative of current fraud patterns – a criticism the literature makes of the benchmark generally [3].

The attributes are anonymised. Because V1–V28 are principal components, no domain-informed feature engineering was possible, and the models’ decisions cannot be explained in terms a cardholder would understand.

Two models were trained on subsampled data. k-NN and the SVM were fitted on reduced training sets for tractability, so their results are not strictly comparable with the models trained on all 198,608 rows. This is declared rather than concealed.

The minority class is small. Only 331 fraudulent transactions were available for training, which limits what any high-capacity model can learn and, as Section 5.5 argues, is the likely reason the trained fusion rule underperformed the fixed one.

6.3 Future work

1. **Resampling techniques.** Apply SMOTE and related methods to address the imbalance at the data level, and compare against the metric-level and threshold-level approaches used here.
2. **Radial Basis Function networks.** The RBF network is covered in Unit IV but was not implemented in this project; it would be a natural addition to the Unit IV comparison.
3. **A larger and more recent dataset.** A benchmark spanning months rather than hours would permit studying how fraud patterns change over time.
4. **Per-transaction cost modelling.** Extend the Bayes optimal decision analysis of Section 4.5 to every model, and make the cost of a missed fraud depend on the transaction amount rather than treating it as constant.
5. **Semi-supervised combination.** Combine the unsupervised clustering of Section 4.9 with the supervised models, so that transactions unlike anything seen before are flagged even when no labelled example resembles them.

References

1. J. O. Awoyemi, A. O. Adetunmbi and S. A. Oluwadare, "Credit card fraud detection using machine learning techniques: A comparative analysis," in *Proc. International Conference on Computing, Networking and Informatics (ICCNi)*, Lagos, Nigeria, IEEE, 2017, pp. 1-9.
2. A. A. Compagnino, Y. Maruccia, S. Cavuoti, G. Riccio, A. Tutone, R. Crupi and A. Pagliaro, "An Introduction to Machine Learning Methods for Fraud Detection," *Applied Sciences*, vol. 15, no. 21, art. 11787, 2025. doi: 10.3390/app152111787.
3. L. Hernandez Aros, L. X. Bustamante Molano, F. Gutierrez-Portela, J. J. Moreno Hernandez and M. S. Rodríguez Barrero, "Financial fraud detection through the application of machine learning techniques: a literature review," *Humanities and Social Sciences Communications*, vol. 11, art. 1130, 2024. doi: 10.1057/s41599-024-03606-0.
4. P. Tiwari, S. Mehta, N. Sakhuja, J. Kumar and A. K. Singh, "Credit Card Fraud Detection using Machine Learning: A Study," *arXiv preprint arXiv:2108.10005*, 2021.
5. D. Varmedja, M. Karanovic, S. Sladojevic, M. Arsenovic and A. Anderla, "Credit Card Fraud Detection — Machine Learning methods," in *Proc. 18th International Symposium INFOTEH-JAHORINA*, IEEE, 2019, pp. 1-5.
6. Machine Learning Group, Université Libre de Bruxelles, "Credit Card Fraud Detection dataset," Kaggle, 2013. [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
7. T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.
8. J. R. Quinlan, "Induction of Decision Trees," *Machine Learning*, vol. 1, no.

- 1, pp. 81–106, 1986.
9. C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
10. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
11. L. Breiman, "Bagging Predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
12. Y. Freund and R. E. Schapire, "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
13. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
14. W. S. McCulloch and W. Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity," *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.
15. F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
16. D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, pp. 533–536, 1986.
17. V. N. Vapnik and A. Ya. Chervonenkis, "On the Uniform Convergence of Relative Frequencies of Events to Their Probabilities," *Theory of Probability and its Applications*, vol. 16, no. 2, pp. 264–280, 1971.
18. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Appendix A: Syllabus Coverage

Every unit of the course unitization plan is applied in the project. The table maps each topic to the notebook that implements it.

Unit	Topic from the unitization plan	Notebook
I	Types of learning; supervised vs unsupervised	01, 09
I	Training, testing and validation of models	01
I	Cross validation	03, 04, 10
I	Overfitting and underfitting	01, 03
I	Evaluation of the model	10
II	Decision tree representation, ID3, entropy	03
II	Hypothesis space search, inductive bias	03
II	Issues in decision tree learning	03
II	Instance based learning, k-nearest neighbour	04
II	Curse of dimensionality	02, 04
II	Univariate feature selection	02
II	Multivariate feature selection	02
II	Feature selection techniques	02
II	Feature reduction: PCA	02
II	Feature reduction: LDA	02
III	Probability and classification, Bayesian learning	05
III	Naive Bayes	05
III	Bayes optimal decisions	05
III	SVM: introduction, dual formulation	06
III	Maximum margin with noise, kernel functions	06
IV	ANN representation, biological motivation	07
IV	McCulloch-Pitts neuron	07
IV	Perceptron, perceptron learning	07

Unit	Topic from the unitization plan	Notebook
IV	Logic gates using perceptron	07
IV	Problem with perceptron (XOR)	07
IV	Gradient descent	07
IV	Multilayer networks, backpropagation	07
IV	Computational learning theory, VC dimension	07
V	Ensembles: bagging	08
V	Random Forest	08
V	Ensembles: boosting	08
V	Fixed rule fusion techniques	08
V	Trained rule fusion techniques	08
V	Clustering: K-means	09
V	Hierarchical clustering	09

Appendix B: Reproducing the Results

6.4 Repository

<https://github.com/lakshay8toic/Credit-Card-Fraud-Detection>

6.5 Steps

```
git clone https://github.com/lakshay8toic/Credit-Card-Fraud-Detection.git
cd Credit-Card-Fraud-Detection
pip install -r requirements.txt
```

```
# place creditcard.csv in Dataset/ (see Dataset/README.md)
jupyter notebook Code/
```

Run the notebooks in order, 01 through 10. Notebook 10 reads the result files written by the earlier notebooks into Results/tables/.

Every stochastic component is seeded with `RANDOM_STATE = 77` in `Code/utils.py`, so re-running reproduces every value in Chapter 5 exactly.

6.6 Output files

Path	Contents
Results/tables/final_comparison.csv	The full results of Table 5.1
Results/tables/cross_validation_comparison.csv	Table 5.3
Results/tables/dataset_summary.json	Dataset properties and split

Path	Contents
Results/tables/syllabus_coverage.csv	Appendix A
Results/tables/feature_methods_comparison.csv	Section 4.2
Results/tables/ensemble_comparison.csv	Section 4.8
Results/figures/	All 61 figures